OCamlFormat

Guillaume Petiot, Tarides

2018-12-13

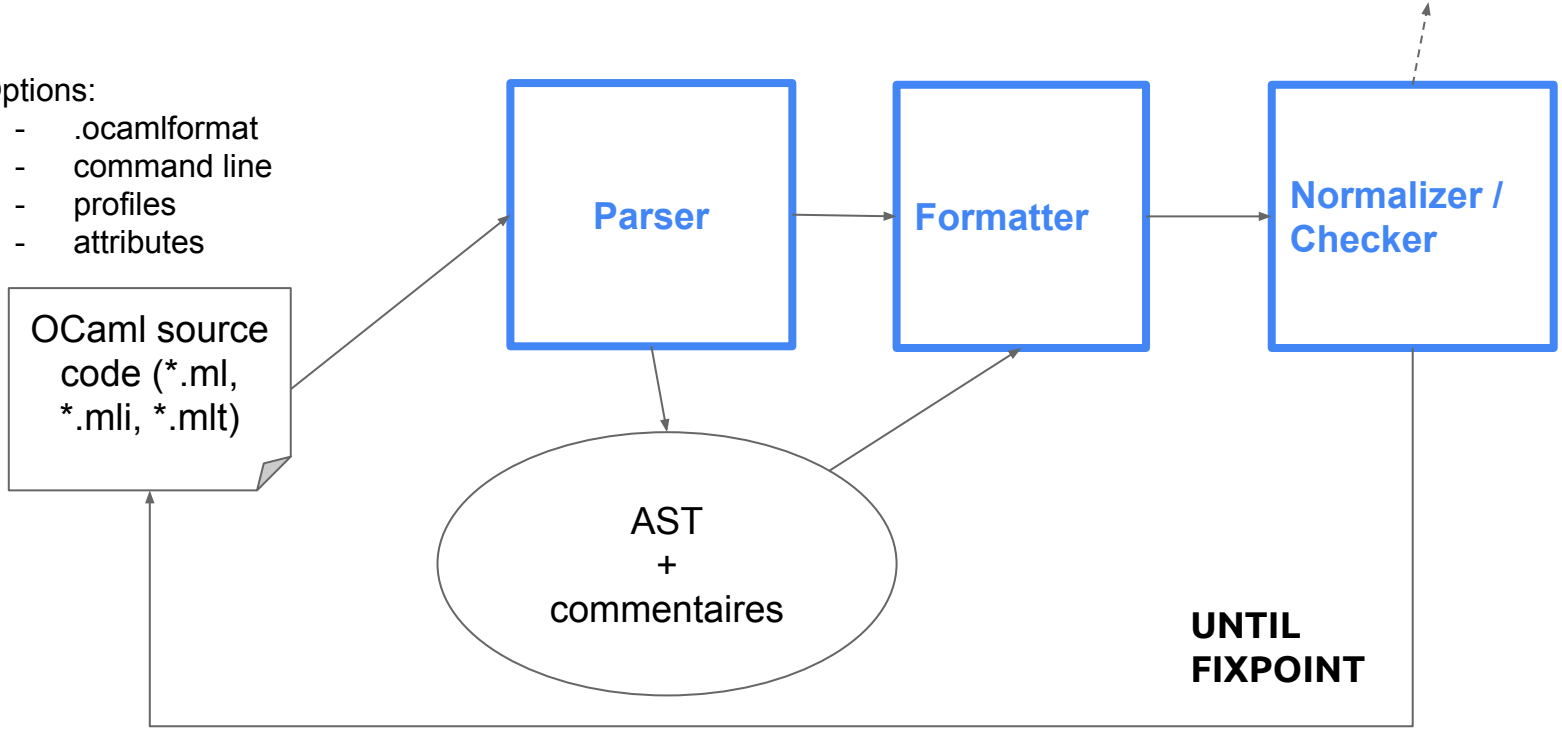
What is it?

- a formatter of OCaml code
- open-source (MIT Licensed)
- **unified formatting style** for the community
- **customizable** to accommodate as many users as possible
  - many options (spaces, linebreaks, parentheses, brackets, wrapping, alignment, etc.)
  - many ways of specifying options (configuration files, command line, environment, OCaml attributes, preset profiles)
- Reason support

How does it work?

Errors:
- AST changed
- comments changed/lost

Options:
- .ocamlformat
- command line
- profiles
- attributes



Examples

--profile = sparse

```
let mk_callback rest name desc = function
  | None ->
      nothing
  | Some f ->
      fun () ->
        ( match rest with
        | [] ->
            f name None
        | (hidden, _) :: _ ->
            f name (Some (desc, hidden)) )
```

--profile = compact

```
let mk_callback rest name desc = function
  | None -> nothing
  | Some f -> (
      fun () ->
        match rest with
        | [] -> f name None
        | (hidden, _) :: _ -> f name (Some (desc, hidden)) )
```

Parsing

```
let foo = (+) bar
```

```
[
  structure_item (/tmp/a.1.ml[1,0+0]..[1,0+19])
        Pstr_value Nonrec
        [
        <def>
        pattern (/tmp/a.1.ml[1,0+4]..[1,0+7])
        Ppat_var "foo" (/tmp/a.1.ml[1,0+4]..[1,0+7])
        expression (/tmp/a.1.ml[1,0+10]..[1,0+19])
        Pexp_apply
        expression (/tmp/a.1.ml[1,0+10]..[1,0+15])
        Pexp_ident "+" (/tmp/a.1.ml[1,0+10]..[1,0+15])
        [
        <arg>
        Nolabel
                expression (/tmp/a.1.ml[1,0+16]..[1,0+19])
                Pexp_ident "bar" (/tmp/a.1.ml[1,0+16]..[1,0+19])
        ]
        ]
]
```

```
(--debug)
```

Formatting

```
[
  structure_item
        Pstr_value Nonrec
        [
        <def>
        pattern                 Ppat_var "foo"
        expression              Pexp_apply
        expression Pexp_ident "+"
        [
        <arg>
        Nolabel
                expression
                Pexp_ident "bar"
        ]
        ]
]
```

```
and fmt_structure_item c ~last:last_item ?ext
{ctx; ast= si} =
      ...
 match si.pstr_desc with
      ...
 | Pstr_value (rec_flag, bindings) ->
        ...
      hvbox 0
      (list_fl grps (fun ~first ~last grp ->
            fmt_grp ~first ~last grp
            $ fmt_if (not last) "\n@\n" ))


...

and fmt_expression c ?(box = true) ?epi ?eol
?parens ?ext ({ast= exp} as xexp) =
      ...
  match pexp_desc with
      ...
  | Pexp_apply (e0, e1N1) ->
      ...
      wrap_if parens "(" ")"
            (hvbox 2 (fmt_args_grouped e0 e1N1)
                        $ fmt_atrs)
```

Formatting with boxes

```
[
    ...
    [
        [let [foo] =]
        [
            [(+)]
            [bar]
        ]
    ]
    ...
]
```

Most of the times boxes match the AST shape

Horizontal box:
`foo, foo, foo`

Vertical box:
```
foo,
foo,
foo
```

Horizontal/vertical box:
- fitting: horizontal
- non-fitting: vertical

Horizontal or vertical box:
- fitting: horizontal
- non-fitting:

```
foo, foo,
foo
```

See the `Format` module

Formatting utility functions

```
val fmt_if_k : bool -> t -> t          (** Conditionally format thunk. *)
val fmt_or_k : bool -> t -> t -> t     (** Conditionally select between two format thunks. *)

val fits_breaks : ?force_fit_if:bool -> ?force_break_if:bool -> string -> string -> t
(** [fits_breaks fits breaks] prints [fits] if the enclosing box fits on one line, and otherwise
prints [breaks], which is a string that optionally starts with a break hint specification such as
["@ "], ["@,"], or ["@;<nspaces offset>"]. *)

val wrap_k : t -> t -> t -> t
(** [wrap_k prologue epilogue body] formats [prologue] then [body] then [epilogue]. *)

val wrap_fits_breaks : ?space:bool -> Conf.t -> string -> string -> t -> t
(** As [wrap], but if [space] is provided, a space is added after prologue and a space hint is
added before epilogue in case the enclosing box breaks. *)
```

How to contribute

Contributions are welcomed

https://github.com/ocaml-ppx/ocamlformat

- Check out the **Good-first-issue**
- Fork, and do your magic
- `make && make test && tools/test_branch.sh <branch>`
- Submit a pull request
- Sign the Facebook CLA

What's next?

- Format ocamldoc comments
- Improvement of the consistency check (comments should not move w.r.t. keywords)
- Mimic ocp-indent options
- More box-breaking formatting: anonymous functions, infix operators
- intermediate representation between AST and formatted form (internship)
    - more flexible with boxes
    - AST transformations

Stay tuned for v0.9

Questions?